LLM2Seq: Repurposing LLM Source Encoders for Encoder-Decoder Summarization with Self-Distilled Multi-Token Prediction

Giang-Bien Kieu[1] **Xuan-Dung Doan**[2]
[1]Hanoi University of Science and Technology, HUST, Vietnam
[2]Viettel Artificial Intelligence Center, Viettel Group, Vietnam
bienkieugiang@gmail.com dungdx4@viettel.com.vn

Abstract

Abstractive summarization is a sequence-to-sequence task. A model reads a document and writes a shorter text. Modern Large Language Model (LLM) checkpoints are good at generation and representation learning. However, many of them do not provide a direct encoder-decoder interface for summarization. Encoder-decoder models fit this task because the encoder reads the source and the decoder writes the summary.

This report presents LLM2Seq. LLM2Seq reuses pretrained LLM source encoders inside an encoder-decoder summarizer. A gated residual adapter maps source states into cross-attention memory. A lightweight Transformer decoder then generates the summary. On WikiLingua, we also study inference speed with self-distilled Multi-Token Prediction (MTP). After the summarizer is trained, MTP heads learn from the frozen decoder. They draft future tokens during decoding. The main decoder verifies these drafted tokens.

We evaluate LLM2Seq on Vietnamese WikiLingua and add a Vietnamese LR-Sum evaluation. The results show that LLM-based source encoders can work as cross-attention memory for summarization. The WikiLingua MTP results also show that speedups should be measured by real throughput, not only by accepted tokens.

Keywords: abstractive summarization; encoder-decoder models; LLM source encoders; adapters; multi-token prediction; Vietnamese WikiLingua; LR-Sum

1 Introduction

1.1 Motivation

Abstractive summarization is a sequence-to-sequence task. The model reads a source document and writes a shorter summary (Sutskever et al., 2014; Bahdanau et al., 2015; Vaswani et al., 2017). Encoder-decoder models fit this structure. The encoder reads the source. The decoder writes the output. Many modern LLMs are decoder-only models (Brown et al., 2020; Dubey et al., 2024; Yang et al., 2024). They generate well, but they mix the source and target in one causal stream. This makes the source-summary boundary less clear.

1.2 Problem Statement

This report asks a simple question. Can pretrained LLM source encoders be reused inside an encoder-decoder summarizer? We study Vietnamese WikiLingua (Ladhak et al., 2020) and add a follow-up evaluation on Vietnamese LR-Sum (Palen-Michel and Lignos, 2023). The goal is to keep strong source representations, expose them through cross-attention, and speed up decoding when possible.

1.3 Gap with Existing Work

T5, BART, and mT5 are strong encoder-decoder models (Raffel et al., 2020; Lewis et al., 2020; Xue et al., 2021). They do not directly reuse newer LLM checkpoints. Recent work shows that decoder-only LLMs can become useful encoders (BehnamGhader et al., 2024; Luo et al., 2025). But summarization still needs a stable bridge from source states to decoder cross-attention.

Context compression methods such as LCLMs compress long prompts into short latent sequences (Li et al., 2026). This helps general long-context inference. It is less natural for source-grounded summarization. Summaries often need names, numbers, steps, and local details. If the source is compressed too early, the decoder may lose these details. LLM2Seq keeps token-level memory instead.

1.4 Contributions

We propose **LLM2Seq**. It is an encoder-decoder summarizer built from repurposed LLM source encoders.

- We adapt LLM source encoders for Vietnamese abstractive summarization on WikiLingua and LR-Sum.
- We build an adapter that turns LLM states into decoder memory.
- On WikiLingua, we train self-distilled MTP heads after the summarizer is trained (Gloeckle et al., 2024; Zhao et al., 2026).
- On WikiLingua, we show that MTP must be judged by real throughput, not only accepted tokens.

2 Background and Related Work

2.1 Encoder-Decoder Summarization

Encoder-decoder models separate reading and writing. The decoder attends to encoder states while generating the summary. This interface comes from attention-based sequence-to-sequence models and Transformers (Sutskever et al., 2014; Bahdanau et al., 2015; Vaswani et al., 2017). T5, BART, and mT5 show that it works well for summarization (Raffel et al., 2020; Lewis et al., 2020; Xue et al., 2021).

2.2 Decoder-only LLMs as Encoders

Decoder-only LLMs are trained for next-token prediction. Their causal mask is good for generation, but less direct for source encoding. LLM2Vec shows that decoder-only LLMs can be converted into text encoders (BehnamGhader et al., 2024). LaMaTE uses LLM encoders for machine translation (Luo et al., 2025). T5Gemma builds encoder-decoder models from modern decoder-only components (Zhang et al., 2025a). LLM2Seq follows this direction for summarization.

2.3 Context Compression and LCLM

LCLMs compress long contexts into short latent sequences for general long-context inference (Li et al., 2026). They optimize compression ratio, memory, and long-context accuracy. LLM2Seq has a different goal. It keeps token-level memory because summaries often need local evidence.

2.4 Efficient Decoding and MTP

Autoregressive decoding emits one token per step. Speculative decoding drafts and verifies tokens (Leviathan et al., 2023). Blockwise decoding and Medusa also try to emit more than one token per step (Stern et al., 2018; Cai et al., 2024). MTP trains heads to predict future tokens (Gloeckle et al., 2024). MTP-D improves these heads with self-distillation (Zhao et al., 2026; Hinton et al., 2015).

2.5 Positioning of LLM2Seq

LLM2Seq is not a context compressor. It is also not a pure prompting method. It asks whether LLM source states can become useful decoder memory for summarization. On WikiLingua, it also tests whether verified MTP gives real speedup (Gloeckle et al., 2024; Zhao et al., 2026). Table 1 compares it with LCLM-style compression (Li et al., 2026).

3 LLM2Seq Method

3.1 Problem Formulation

Let $x = (x_1, \ldots, x_m)$ be the source document and $y = (y_1, \ldots, y_n)$ be the target summary. Let $a_i^x \in \{0, 1\}$ and $a_t^y \in \{0, 1\}$ be the source and target masks. The model estimates:

$$p(y|x) = \prod_{t=1}^{n} p(y_t \mid y_{<t}, x). \tag{1}$$

LLM2Seq makes the conditioning path explicit. The source encoder builds token states:

$$H = E_\theta(x), \qquad H \in \mathbb{R}^{m \times d_e}. \tag{2}$$

The adapter maps them into decoder memory:

$$M = A_\phi(H), \qquad M \in \mathbb{R}^{(m+g) \times d_d}, \tag{3}$$

where g is the number of learned global memory tokens. This memory M is the source-memory block shown in Figure 1. The decoder then generates:

$$p(y|x) = \prod_{t=1}^{n} p_\psi(y_t \mid y_{<t}, M). \tag{4}$$

Training uses teacher forcing. With non-padding target positions $\Omega = \{t : a_t^y = 1\}$, the summarization loss is:

$$\mathcal{L}_{sum} = -\frac{1}{|\Omega|} \sum_{t \in \Omega} \log p_\psi(y_t \mid y_{<t}, M). \tag{5}$$

Aspect	LCLM / End-to-End Context Compression	LLM2Seq
Main goal	Compress long context for general inference	Vietnamese abstractive summarization
Source representation	Short latent compressed sequence	Token-level cross-attention memory
Compression ratio	Explicit optimization target	Not the main objective
Output setting	General long-context inference	Encoder-decoder summary generation
Speed mechanism	Shorter context and reduced memory	Verified MTP decoding
Risk	Information loss from compression	More memory and cross-attention cost
Best use case	Long redundant context	Source-grounded summarization

Table 1: Conceptual comparison with LCLM-style context compression. This is not an empirical LCLM baseline.

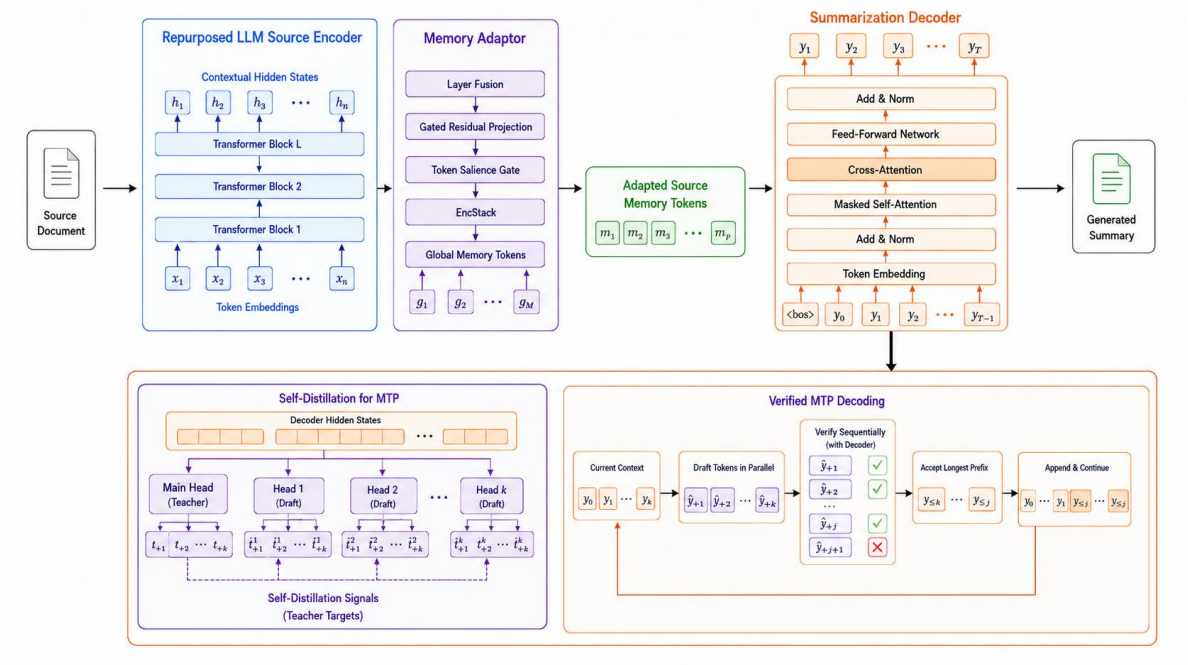

Figure 1: Overall LLM2Seq architecture with the source encoder, memory adapter, decoder, and MTP modules.

This form is the core design choice. The encoder handles source understanding. The decoder handles target generation. The adapter decides how source evidence is exposed to cross-attention.

3.2 Overall Architecture

Figure 1 shows the full system. LLM2Seq has three main parts. First, the system uses a pretrained LLM-based source encoder. We instantiate it with either an LLM2Vec-derived Llama encoder or a Qwen embedding encoder (BehnamGhader et al., 2024; Zhang et al., 2025b). Second, a gated residual memory adapter reshapes the encoder states into decoder memory. The adapter section later expands this memory as learned global tokens plus adapted token-level source states. In Figure 1, *memory adapter* refers to this whole bridge. It is not a separate summarizer. It converts LLM hidden states into the memory that the decoder reads through cross-attention. Third, a lightweight Transformer decoder generates the summary with masked self-attention and cross-attention (Vaswani et al., 2017). After the summarizer is trained, an MTP module is attached for acceleration. The MTP module is trained separately and verified by the main decoder during inference.

3.3 Repurposed LLM Source Encoder

We use two source encoders. The Llama variant uses an LLM2Vec Sheared-LLaMA checkpoint (BehnamGhader et al., 2024). The Qwen variant uses Qwen3-Embedding-0.6B (Zhang et al., 2025b). This distinction matters for the scope of the claim: LLM2Seq studies reusable LLM source encoders, not only pure decoder-only checkpoints. For a source sequence x, the encoder returns token-level hidden states:

$$H^{(\ell)} = F_\theta^{(\ell)}(x, a^x), \qquad H^{(\ell)} \in \mathbb{R}^{m \times d_e}. \quad (6)$$

Unlike sentence embedding use cases, LLM2Seq keeps the full sequence of token states. This is needed because the decoder must attend to local

source evidence. The encoder also returns a set of selected layers:

$$\mathcal{H}_S = \{H^{(l_1)}, H^{(l_2)}, \ldots, H^{(l_s)}\}. \quad (7)$$

In our setting, the selected layers are $[-1, -4, -8, -12]$. Full fine-tuning is expensive. Phase 2 therefore applies LoRA to the encoder (Hu et al., 2022). For a frozen encoder weight matrix W, LoRA learns a low-rank update:

$$W' = W + \frac{\alpha}{r} BA, \quad (8)$$

where $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$. Only the low-rank matrices are updated for the encoder. This adapts the source representation to Vietnamese summarization while keeping training practical.

3.4 Gated Residual Memory Adapter

The adapter is the bridge between the LLM encoder and the decoder. It must solve a mismatch. The encoder states live in the LLM representation space, while the decoder expects memory in its own hidden space. The adapter also needs to keep token-level evidence, because summaries often depend on details in the source. The overall adapter composition is specific to LLM2Seq. However, its individual mechanisms are not claimed as new neural primitives. Some are direct applications of standard Transformer components. Others adapt layer mixing, gated updates, feature modulation, and learned prompt tokens to the source-memory interface.

First, token-wise layer fusion combines selected encoder layers. This is inspired by learned layer mixing in contextual representation models such as ELMo (Peters et al., 2018). The adaptation in LLM2Seq is token-wise rather than document-global: each source token can choose a different mixture over selected LLM layers. For token position i and selected layer j, a score is computed by:

$$e_{ij} = w_f^\top H_i^{(l_j)}. \quad (9)$$

The layer mixture is:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{r=1}^{s} \exp(e_{ir})}, \qquad \bar{H}_i = \sum_{j=1}^{s} \alpha_{ij} H_i^{(l_j)}. \quad (10)$$

This lets different tokens use different depths of the LLM. Lexical cues and semantic cues do not always appear at the same layer.

The fused states are then projected into decoder space. We use a gated residual projection for this step. Gated residual mechanisms are widely used to modulate feature updates. Examples include Highway Networks and gated linear units (Srivastava et al., 2015; Dauphin et al., 2017). LLM2Seq does not claim this gating equation as a new neural block. Instead, we apply it to a specific interface problem. It bridges a frozen or lightly adapted LLM source encoder and a smaller cross-attentive decoder. The implemented gated residual projection first normalizes the fused state:

$$\hat{H} = \mathrm{LN}(\bar{H}). \quad (11)$$

It builds a base path, a gate, and an update:

$$P = W_p \hat{H} + b_p, \qquad G = \sigma(W_g \hat{H} + b_g), \quad (12)$$

$$U = W_2\, \mathrm{GELU}(W_1\, \mathrm{LN}(P) + b_1) + b_2. \quad (13)$$

The adapted token state is:

$$R = P + G \odot U. \quad (14)$$

The base path P maps the encoder states into the decoder dimension. The update block U adds task-specific changes. The gate G controls how much of this update is used. This residual form keeps the original source information easier to preserve while still allowing the adapter to add summarization-specific corrections.

A token salience gate then estimates which source positions should be emphasized. This gate is inspired by feature recalibration and feature-wise conditioning mechanisms such as Squeeze-and-Excitation and FiLM (Hu et al., 2018; Perez et al., 2018). LLM2Seq adapts this idea to token-level source memory. The goal is not to remove tokens, but to rescale their representations before decoder cross-attention.

$$S = \sigma(W_{s2}\, \mathrm{GELU}(W_{s1}\, \mathrm{LN}(R) + b_{s1}) + b_{s2}). \quad (15)$$

Our Token Salience Gate differs from token pruning and token selection methods. Instead of suppressing or discarding low-salience tokens, it performs continuous feature rescaling:

$$R'_i = (0.5 + S_i) R_i, \quad (16)$$

ensuring every token contributes while allowing salient representations to be emphasized. Padding positions have $S_i = 0$. Thus padded memory cannot become salient.

An EncStack refines the memory with $L_a = 4$ pre-norm Transformer encoder layers (Vaswani et al., 2017). This part is a direct application of standard Transformer encoder refinement. In LLM2Seq, it is placed after the projection and salience gate so that adapted source tokens can interact before they are read by the decoder.

$$\bar{R}^{(0)} = R', \tag{17}$$

$$Q^{(u)} = \bar{R}^{(u-1)} + \text{SelfAttn}(\text{LN}(\bar{R}^{(u-1)}), a^x), \tag{18}$$

$$\bar{R}^{(u)} = Q^{(u)} + \text{FFN}(\text{LN}(Q^{(u)})). \tag{19}$$

Let $\tilde{R} = \bar{R}^{(L_a)}$. The final memory concatenates learned global memory tokens G_m with the refined token memory. Learned global tokens are inspired by prefix tuning and prompt tuning (Li and Liang, 2021; Lester et al., 2021). In those methods, trainable continuous tokens condition a language model. LLM2Seq uses them differently. They are decoder-visible source-memory slots trained for summarization, not a prompt prepended to the decoder input.

$$M = [G_m;\, \tilde{R}]. \tag{20}$$

The corresponding memory mask is:

$$a^M = [\mathbf{1}_g;\, a^x]. \tag{21}$$

The global tokens give the decoder a small document-level workspace. The token memory gives the decoder direct access to local evidence. Together, this concatenated sequence M forms the *adapted source memory tokens*. In the rest of this report, *adapted source memory* refers to this sequence $M = [G_m; \tilde{R}]$. It contains global memory tokens followed by adapted token-level source states. It provides the decoder with both document-level context and fine-grained local details in a unified representation space.

3.5 Lightweight Cross-Attentive Decoder

The decoder is an 8-layer Transformer decoder (Vaswani et al., 2017). It uses causal self-attention, cross-attention to M, RoPE, RMSNorm, and SwiGLU (Su et al., 2021). At step t, it reads $y_{<t}$ and the adapted source memory. The next-token distribution is:

$$\begin{aligned} o_t &= E^\top \text{RMSNorm}(Y_t^{(L_d)}), \\ p_\psi(y_t \mid y_{<t}, M) &= \text{softmax}(o_t). \end{aligned} \tag{22}$$

The decoder is intentionally smaller than the encoder. This keeps generation cheaper while still allowing the encoder to carry strong source representations.

3.6 Self-Distilled MTP

After Phase 2, the main summarizer is frozen. Phase 3 adds $K = 4$ cascaded MTP heads (Gloeckle et al., 2024; Zhao et al., 2026; DeepSeek-AI, 2024). The heads predict future tokens beyond the next token. The loss follows MTP-D style self-distillation. The frozen main head acts as the teacher (Zhao et al., 2026; DeepSeek-AI, 2024). LLM2Seq uses this idea after summarizer training. The encoder, adapter, decoder, and main LM head are frozen. Only the MTP heads are updated. This makes MTP a post-hoc speed module, not a joint pre-training objective. Let $h_t^{(0)} = Y_t^{(L_d)}$ be the frozen decoder state at target position t. The main head predicts:

$$p_t^{(0)} = \text{softmax}(E^\top h_t^{(0)}). \tag{23}$$

Each MTP depth k predicts y_{t+k}:

$$p_t^{(k)} = \text{softmax}(E^\top h_t^{(k)}), \qquad k = 1, \ldots, K. \tag{24}$$

The hard-label loss trains the heads on gold future tokens:

$$\mathcal{L}_{CE} = -\frac{1}{|\Omega_k|} \sum_{k=1}^{K} w_k \sum_{t \in \Omega_k} \log p_t^{(k)}(y_{t+k}). \tag{25}$$

The KL loss makes each MTP head follow the frozen main head at the same future position (Hinton et al., 2015). We compute it on the top-J teacher logits:

$$\mathcal{L}_{KL} = \frac{\tau^2}{\sum_k w_k} \sum_{k=1}^{K} w_k \frac{1}{|\Omega_k|} \sum_{t \in \Omega_k} \text{KL}(q_{t,k} \,\|\, s_{t,k}). \tag{26}$$

The final Phase 3 loss is:

$$\mathcal{L}_{MTP} = \lambda_{CE}\mathcal{L}_{CE} + \gamma(u)\lambda_{KL}\mathcal{L}_{KL}, \tag{27}$$

Here $\gamma(u)$ ramps in the KL term during training. Only the MTP heads are updated in this phase.

3.7 Verified MTP Decoding Algorithm

At inference time, the MTP heads draft future tokens. The main decoder verifies them. This follows the draft-and-verify idea in speculative decoding and MTP-style decoding (Leviathan et al.,

2023; Gloeckle et al., 2024; Zhao et al., 2026). Let $g(\pi)$ be the constrained greedy main-head token for prefix π. It includes the same repetition penalty, minimum length rule, and no-repeat trigram constraint as standard decoding. For a current prefix π, the main decoder first predicts $c_0 = g(\pi)$. The MTP module then drafts future tokens:

$$c_k = \arg\max_v p^{(k)}(v \mid \pi, c_{<k}), k = 1, \ldots, K. \tag{28}$$

The candidate block is $c = (c_0, c_1, \ldots, c_K)$. The verifier runs the frozen decoder on this block. It accepts the longest draft prefix that matches the main decoder. The first mismatch is corrected by the verifier token. Algorithm 1 gives the procedure.

Algorithm 1 Verified MTP Draft-and-Check Decoding

1: Encode the source once and cache cross-attention keys and values.
2: **repeat**
3: Predict the main token c_0 from the current prefix.
4: Draft K future tokens with the cascaded MTP heads.
5: Verify the candidate block with the frozen main decoder.
6: Accept the longest draft prefix that matches the verifier.
7: Emit the accepted prefix and the verifier correction token.
8: Slice the decoder cache to the verified prefix.
9: **until** the end token or the maximum length is reached

If step s emits e_s tokens, the average emitted tokens per step is:

$$\bar{e} = \frac{\sum_s e_s}{T}, \tag{29}$$

where T is the number of verification steps. A high $\bar{e}$ reduces the number of steps. It does not guarantee higher throughput because verification also costs time.

3.8 Training Procedure

LLM2Seq uses three phases. Phase 1 freezes the encoder and trains the adapter, decoder, global memory tokens, and LM head with $\mathcal{L}_{sum}$. Phase 2 adds LoRA to the encoder and continues optimizing the summarization loss (Hu et al., 2022). Phase 3 freezes the main summarizer and trains only the MTP module with $\mathcal{L}_{MTP}$. This ordering matters. The summarizer is built first. The speed module is trained after that. Thus Phase 3 is a decoupled, post-hoc use of MTP for inference speed. It is not the joint MTP pre-training objective from the original MTP papers (Gloeckle et al., 2024; Zhao et al., 2026; DeepSeek-AI, 2024).

3.9 Complexity and Inference Cost

The encoder runs once per source document. The autoregressive decoder then runs once per generated token. Let C_{AR} be the average cost of one autoregressive decoding step. If a summary has n generated tokens, the decoding cost is roughly:

$$C^{AR}_{\text{decode}} \approx nC_{\text{AR}}. \tag{30}$$

Verified MTP emits $\bar{e}$ tokens per step on average, but each step has extra draft and verification cost C_{ver}:

$$C^{MTP}_{\text{decode}} \approx \frac{n}{\bar{e}}(C_{\text{AR}} + C_{\text{ver}}). \tag{31}$$

MTP is faster only when:

$$\bar{e} > 1 + \frac{C_{\text{ver}}}{C_{\text{AR}}}. \tag{32}$$

This explains why accepted tokens and real throughput can disagree. The system must save more decoder steps than it adds in verification overhead.

4 Experimental Setup

4.1 Dataset and Preprocessing

We evaluate on the Vietnamese subset of WikiLingua, a multilingual abstractive summarization dataset extracted from WikiHow articles (Ladhak et al., 2020). Both source articles and summaries are Vietnamese. The original WikiLingua release reports 19,600 Vietnamese article-summary pairs. Our local split contains 19,581 raw records. Preprocessing removes one test example with an empty source, leaving 19,580 examples. Table 2 summarizes the split.

We also evaluate on the Vietnamese split of LR-Sum (Palen-Michel and Lignos, 2023). LR-Sum is a multilingual news summarization dataset for less-resourced languages, with human-written summaries from Voice of America newswire. The Vietnamese split contains 14,595 examples in our processed copy, as shown in Table 3. For LR-Sum, we use the article text as the source and the summary field as the target.

Split	Raw	Used
Train	13,999	13,999
Validation	1,680	1,680
Test	3,902	3,901
Total	19,581	19,580

Table 2: Vietnamese WikiLingua data split.

Split	Raw	Used
Train	11,676	11,676
Validation	1,459	1,459
Test	1,460	1,460
Total	14,595	14,595

Table 3: Vietnamese LR-Sum data split.

4.2 Compared Systems and Baselines

We compare three systems. **LLM2Seq-Llama** uses the LLM2Vec-Sheared-LLaMA encoder (BehnamGhader et al., 2024). **LLM2Seq-Qwen** uses the Qwen3-Embedding-0.6B encoder (Zhang et al., 2025b). Both use the same adapter and lightweight decoder design. **T5Gemma** is a T5Gemma-2-1B-1B LoRA baseline (Zhang et al., 2025a; Hu et al., 2022).

4.3 Implementation Details

The WikiLingua source length is up to 3,072 tokens and the target length is up to 384 tokens during training. For LR-Sum, the source length remains 3,072 tokens and the target length is 256 tokens. Generation uses at most 256 new tokens. All reported systems use greedy decoding and no-repeat trigram blocking. LLM2Seq uses bf16/tf32 training. Both LLM2Seq variants adapt the encoder with LoRA (Hu et al., 2022). For the WikiLingua experiments, the MTP module uses four heads and is trained after the main summarizer is frozen. We use AdamW with betas $(0.9, 0.95)$ and $\epsilon = 10^{-8}$, a linear warmup, cosine decay, and gradient clipping at 1.0. For WikiLingua, Phase 1 trains the adapter and decoder for 6 epochs. It uses batch size 24, gradient accumulation 2, learning rate $2 \cdot 10^{-4}$, warmup ratio 0.03, and weight decay 0.01. Phase 2 adapts the encoder with LoRA. For WikiLingua, Phase 2 runs for 6 epochs. It uses batch size 16, gradient accumulation 2, encoder learning rate $5 \cdot 10^{-4}$, decoder and adapter learning rate $5 \cdot 10^{-5}$, and weight decay 0.05. Both WikiLingua LLM2Seq runs also use gradual unfreezing. On WikiLingua, Phase 3 freezes the encoder, adapter, and decoder. It trains only the MTP heads for 6 epochs. It uses batch size 20, gradient accumulation 4, learning rate $5 \cdot 10^{-4}$, warmup ratio 0.05, and weight decay 0.01. For LR-Sum, the LLM2Seq runs cover Phase 1–2 quality evaluation only. They do not include Phase 3 MTP. The LR-Sum Llama runs use 3 epochs for Phase 1 and 3 epochs for Phase 2; LR-Sum Qwen uses 3 epochs for Phase 1 and 6 epochs for Phase 2. Checkpoints are selected by validation loss. For generation, all WikiLingua models use a minimum of 32 new tokens and repetition penalty 1.15. For LR-Sum, the available runs use a minimum of 32 new tokens and repetition penalty 1.05. The WikiLingua T5Gemma baseline is trained for 6 epochs with LoRA. It uses batch size 16, gradient accumulation 2, learning rate $2 \cdot 10^{-4}$, warmup ratio 0.03, weight decay 0.01, cosine schedule, and fused AdamW. The LR-Sum T5Gemma run is trained for 3 epochs. Appendix A summarizes the training and decoding configuration.

4.4 Evaluation Metrics

We report ROUGE (Lin, 2004), BERTScore (Zhang et al., 2020), length statistics, and speed. ROUGE-1 and ROUGE-2 measure unigram and bigram overlap. ROUGE-L measures longest common subsequence overlap. We report F1 for all ROUGE scores. For Vietnamese text, ROUGE is computed with the Python `rouge-score` implementation using `use_stemmer=False`. Decoded strings are evaluated directly. Length statistics use whitespace-separated tokens from `text.split()`, which we call words for readability. We do not apply Vietnamese word segmentation such as VnCoreNLP or underthesea. This matters because Vietnamese word segmentation can change overlap scores. For length, we report mean predicted words and too-long rate. The too-long rate is the percentage of predictions whose word count is more than 1.5 times the reference word count. BERTScore is computed with `bert-base-multilingual-cased`. It matches generated and reference tokens by contextual embedding similarity. We report precision, recall, and F1 as percentages. BERTScore is more semantic than ROUGE, but it is not a factuality metric. All BERTScore values reported in the main result tables use `bert-base-multilingual-cased`. For speed, we report generated tokens per second and mean latency. We compare speed only when the batch setting is the same. When comparing AR

and MTP speed on WikiLingua, we use paired comparison logs under the same model and batch setting.

4.5 Research Questions

We evaluate LLM2Seq with three questions. They cover summarization quality, encoder configuration, and decoding speed.

- **RQ1**: Can pretrained LLM-based source encoders be reused inside an encoder-decoder summarizer?
- **RQ2**: On WikiLingua, how much do the LLM2Seq encoder configurations affect quality, length, and speed?
- **RQ3**: On WikiLingua, does verified MTP improve real throughput, or does it only reduce decoding steps?

5 Results and Analysis

5.1 RQ1: Viability of LLM-based Source Encoders

Table 4 reports WikiLingua and LR-Sum test results. On WikiLingua, the Qwen-based LLM2Seq system achieves 54.01 ROUGE-1, 20.83 ROUGE-2, 31.83 ROUGE-L, and 73.42 BERTScore F1. The WikiLingua result shows that a pretrained LLM-based source encoder can provide useful cross-attention memory for an encoder-decoder summarizer. On WikiLingua, the Llama-based system also produces valid summaries. Its ROUGE scores are lower, and its outputs are much shorter. This shows that the encoder configuration matters.

On WikiLingua, LLM2Seq-Qwen is close to T5Gemma on ROUGE-1. It is lower on ROUGE-2 and ROUGE-L. This supports the LLM2Seq design, but it does not beat T5Gemma. The result also depends on the encoder choice and output length.

On LR-Sum, LLM2Seq-Qwen improves over LLM2Seq-Llama. The gain is clearest on ROUGE-2 and BERTScore F1. Its average output length is also close to the reference length. T5Gemma is still much stronger on ROUGE-2 and ROUGE-L. The LR-Sum LLM2Seq outputs are fluent, but they can choose wrong entities or events. This lowers n-gram overlap with the reference.

5.2 RQ2: Effect of Encoder Configuration on WikiLingua

On WikiLingua, the two LLM2Seq variants behave differently. LLM2Seq-Llama undergenerates. Its average output length is 29.10 words, far below the 51.86-word reference average. LLM2Seq-Qwen generates longer summaries. Its average output length is 58.31 words, which is closer to the reference. This length difference partly explains the large ROUGE gap between the two LLM2Seq variants.

The WikiLingua comparison should be read as a controlled configuration-level comparison rather than a pure encoder-only ablation. Both variants use the same overall LLM2Seq architecture, adapter design, decoder design, and decoding constraints. However, the source encoder choice also changes the tokenizer, vocabulary, and representation space.

Table 5 shows WikiLingua autoregressive decoding speed. LLM2Seq-Llama is faster than LLM2Seq-Qwen: 114.74 tokens/s versus 95.87 tokens/s. A likely explanation is the decoder-side vocabulary size. Because the decoder uses a tied input/output embedding matrix, the final LM head depends on the tokenizer vocabulary. The larger Qwen vocabulary can make each decoding step more expensive. At the same time, Qwen gives better summarization quality. This gives a trade-off on WikiLingua. Qwen improves quality and length control. Llama is faster, but it undergenerates.

5.3 RQ3: Verified MTP Throughput on WikiLingua

Table 6 compares autoregressive decoding and verified MTP for the Qwen-based system on WikiLingua. Verified MTP increases the emitted tokens per decoding step from 1.00 to 2.30 and improves throughput from 95.87 tokens/s to 111.71 tokens/s. This gives a measured speedup of 1.17x and reduces mean latency from 0.779s to 0.662s.

However, the speedup is much smaller than the emitted-token-per-step increase. This suggests that accepted or emitted tokens alone are not enough to evaluate MTP. The verifier also costs time. Cache handling and tensor control flow also add overhead. For this reason, we report both speed and quality for AR and MTP. Verified MTP is designed to follow the main decoder path. Even so, AR and MTP can still produce small metric

Dataset	Model	R-1	R-2	R-L	BERT-P	BERT-R	BERT-F1	Avg. Words	Too Long (%)
WikiLingua	LLM2Seq-Llama	48.36	15.54	29.05	**74.08**	72.11	73.02	29.10	5.46
	LLM2Seq-Qwen	54.01	20.83	31.83	72.87	74.11	73.42	58.31	35.79
	T5Gemma	**54.24**	**27.42**	**33.89**	71.99	**78.40**	**74.98**	90.89	68.03
LR-Sum	LLM2Seq-Llama	50.49	12.08	29.41	70.74	69.26	69.98	26.78	4.18
	LLM2Seq-Qwen	51.59	15.58	31.21	72.24	71.04	71.62	29.50	6.92
	T5Gemma	**58.79**	**27.34**	**39.86**	**74.13**	**75.24**	**74.65**	33.73	14.18

Table 4: Test results on WikiLingua and Vietnamese LR-Sum. Scores are percentages. BERTScore uses `bert-base-multilingual-cased` without baseline rescaling. **Avg. Words** is mean output length. **Too Long** is the percentage of predictions with more than 1.5x the reference word count.

Model	tok/s	Mean latency
LLM2Seq-Llama	114.74	0.697s
LLM2Seq-Qwen	95.87	0.779s

Table 5: Autoregressive decoding speed for LLM2Seq variants on WikiLingua under the same batch setting.

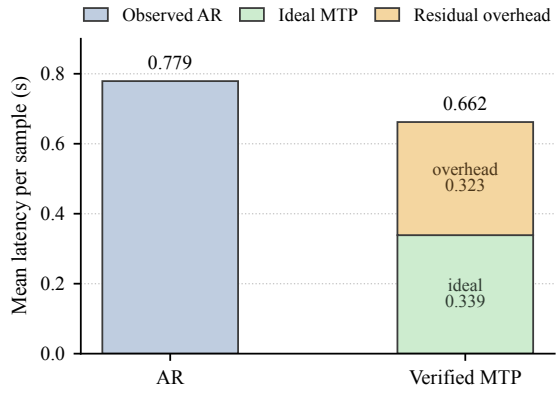

Figure 2: Latency accounting for Qwen AR and verified MTP on WikiLingua. The ideal component divides AR latency by emitted tokens per step. Residual is the remaining latency.

differences. They use different cache handling, stopping behavior, and decoding-control code. We therefore report quality scores for both modes.

Figure 2 shows a derived latency accounting for the WikiLingua Qwen system. If each MTP step cost the same as an autoregressive step, emitting 2.30 tokens per step would reduce mean latency from 0.779s to about 0.339s. In practice, the observed MTP latency is 0.662s, leaving about 0.323s as residual overhead. This is not kernel-level profiling. It is a conservative estimate from the evaluation logs. It points to verification, cache management, tensor control flow, and Python overhead as likely bottlenecks.

For the Qwen WikiLingua system, verified MTP improves real throughput. The gain is modest. MTP speed still depends on implementation details.

5.4 Comparison with End-to-End Context Compression

We do not claim a direct empirical comparison with LCLMs. The current project does not include an LCLM baseline. The comparison is conceptual. As summarized in Table 1, LCLMs compress contexts into shorter latent sequences for general long-context inference (Li et al., 2026). LLM2Seq keeps token-level source memory and trains for Vietnamese summarization. This distinction matters. LLM2Seq does not optimize compression ratio. It studies whether token-level LLM states can work as source memory for summarization. A fair future comparison should add block compression baselines, such as mean-pooling encoder states by 4, 8, or 16 tokens before the decoder.

5.5 Qualitative Example

Appendix C gives examples from the WikiLingua test prediction files. It shows the same pattern as the aggregate results. LLM2Seq-Llama is short but loses key content. LLM2Seq-Qwen often keeps the broad topic and a concise style, but it can also make serious semantic errors. T5Gemma is more fluent and covers more details, but it is also longer. This WikiLingua example matches the table. Different LLM2Seq configurations choose different content and output lengths.

5.6 Error Analysis

The WikiLingua results reveal three error patterns. First, on WikiLingua, LLM2Seq-Llama undergenerates, which hurts recall and leads to lower ROUGE. Second, on WikiLingua, T5Gemma often generates much longer summaries. This can improve overlap scores but reduces conciseness. Third, on WikiLingua, verified MTP improves

Model	Mode	tok/s	Spd.	Lat.	Emit	Draft	R-1	R-2	R-L	BERT-F1
Llama	AR	114.74	1.00x	0.697	1.00	0.00	48.36	15.54	29.05	73.02
Qwen	AR	95.87	1.00x	0.779	1.00	0.00	54.01	20.83	31.83	73.42
Qwen	MTP	**111.71**	**1.17x**	**0.662**	2.30	0.44	53.90	20.81	31.64	73.26

Table 6: WikiLingua AR and verified MTP results. Lat. is mean latency. Emit is emitted tokens per step. Draft is accepted draft tokens per step.

throughput only modestly. It emits multiple tokens per step, but the verifier still costs time.

6 System Demonstration

We implement a web interface for running LLM2Seq summarization. The interface loads the Qwen-based model, accepts a source document, and shows the generated summary. It supports two modes. Autoregressive mode generates one token at a time. Verified MTP mode drafts future tokens and verifies them with the frozen decoder. The demo reports latency, generated tokens, tokens per second, and accepted-token statistics. Appendix D shows the interface. The demo is for inspection. The main evidence comes from the evaluation logs.

7 Discussion

LLM2Seq shows that LLM-based source encoders can be reused inside an encoder-decoder summarizer. On WikiLingua, the Qwen variant is close to T5Gemma on ROUGE-1. It also produces shorter summaries. This supports the design. It does not show that LLM2Seq is better than standard encoder-decoder baselines. On WikiLingua, the Qwen-based LLM2Seq variant achieves higher ROUGE than the Llama-based variant. This should be read as a system-level comparison. The tokenizer, vocabulary, and representation space also change. For Qwen on WikiLingua, MTP gives 2.30 tokens per step but only 1.17x real speedup. MTP helps only when accepted tokens pay for verification. Token-level memory is useful when the summary needs local source evidence. This differs from fixed-ratio context compression (Li et al., 2026). The current verified decoder is an implementation-level prototype. Future deployment should fuse verification, reduce Python control flow, and use optimized runtimes such as vLLM. Quantization methods such as SmoothQuant may also help under memory limits (Xiao et al., 2023).

8 Threats to Validity

There are five main threats. First, the main experiments use Vietnamese WikiLingua (Ladhak et al., 2020). LR-Sum (Palen-Michel and Lignos, 2023) is only an additional Phase 1–2 quality evaluation. The result may not transfer to dialogue, legal text, scientific text, or other summarization domains. Second, the report uses T5Gemma as the main baseline. It does not include many other baselines such as mT5, viT5, mBART, or prompted LLMs. Third, Llama and Qwen differ in tokenizer, vocabulary, and representation space. So their comparison is not a pure encoder-only ablation. Fourth, ROUGE and BERTScore do not fully measure factuality or usefulness. They should be read with qualitative examples. Fifth, the WikiLingua MTP speed result depends on the current implementation. Verifier cost, cache handling, and Python control flow can reduce speedup.

9 Conclusion

LLM2Seq studies how to build an encoder-decoder summarizer from repurposed LLM source encoders. The model uses a gated residual adapter to map source-side LLM states into decoder memory. It then uses a lightweight cross-attentive decoder to generate summaries. On Vietnamese WikiLingua, the Qwen-based variant is competitive with T5Gemma on ROUGE-1 and produces shorter summaries. On Vietnamese LR-Sum, LLM2Seq-Qwen improves over LLM2Seq-Llama but still trails T5Gemma. Self-distilled MTP gives a real 1.17x speedup for Qwen on WikiLingua. Future work should add adapter ablations, add compression baselines, and optimize verified decoding.

Limitations

This report does not include a full adapter ablation. It cannot isolate the contribution of layer fusion, salience gating, EncStack refinement, or global memory tokens. It also does not include an LCLM-style block-compression baseline. The compari-

son with context compression is therefore conceptual. The evaluation relies on automatic metrics and does not include human factuality judgments. Finally, the current demo is an engineering artifact for inspection, not evidence for user-facing reliability.

Ethics Statement

This work uses an existing public summarization dataset and does not introduce human-subject data collection. The system may still produce incomplete or incorrect summaries. It should not be used as the only source of information in high-stakes settings.

Broader Impact

LLM2Seq may make summarization systems easier to build from existing LLM checkpoints. This can reduce training cost and make summarization more accessible for lower-resource languages. At the same time, better summarization tools can also spread errors faster if users over-trust generated summaries.

Code Availability

The repository includes the implementation, evaluation scripts, verified MTP decoding code, and the demo interface: `github.com/BienKieu1411/LLM2Seq`.

Acknowledgments

We thank the project reviewers for feedback on framing, experimental reporting, and report structure.

References

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations*.

Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. LLM2Vec: Large language models are secretly powerful text encoders. In *First Conference on Language Modeling*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*.

Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. 2017. Language modeling with gated convolutional networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 933–941.

DeepSeek-AI. 2024. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Roziere, David Lopez-Paz, and Gabriel Synnaeve. 2024. Better and faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Jie Hu, Li Shen, and Gang Sun. 2018. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7132–7141.

Faisal Ladhak, Esin Durmus, Claire Cardie, and Kathleen McKeown. 2020. WikiLingua: A new benchmark dataset for cross-lingual abstractive summarization. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4034–4048, Online. Association for Computational Linguistics. Dataset release: `https://github.com/esdurmus/Wikilingua`.

Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer.

2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7871–7880.

Ang Li, Sean McLeish, Haozhe Chen, Nimit Kalra, Zaiqian Chen, Artem Gazizov, Venkata Anoop Suhas Kumar Morisetty, Bhavya Kailkhura, Harshitha Menon, Zhuang Liu, Brian R. Bartoldson, Tom Goldstein, Sanae Lotfi, Micah Goldblum, and Pavel Izmailov. 2026. End-to-end context compression at scale. *arXiv preprint arXiv:2606.09659*.

Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, pages 4582–4597.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81.

Yingfeng Luo, Tong Zheng, Yongyu Mu, Bei Li, Qinghong Zhang, Yongqi Gao, Ziqiang Xu, Peinan Feng, Xiaoqian Liu, Tong Xiao, and Jingbo Zhu. 2025. Beyond decoder-only: Large language models can be good encoders for machine translation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 9399–9431.

Chester Palen-Michel and Constantine Lignos. 2023. LR-sum: Summarization for less-resourced languages. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 6829–6844, Toronto, Canada. Association for Computational Linguistics.

Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. 2018. FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2227–2237.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. 2015. Highway networks. *arXiv preprint arXiv:1505.00387*.

Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. 2018. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems*.

Jianlin Su, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*.

Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*.

Linting Xue, Noah Constant, Adam Roberts, Mihir Kale, Rami Al-Rfou, Aditya Siddhant, Aditya Barua, and Colin Raffel. 2021. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 483–498.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, and 1 others. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.

Biao Zhang, Paul Suganthan, Gael Liu, Ilya Philippov, Sahil Dua, Ben Hora, Kat Black, Gus Martins, Omar Sanseviero, Shreya Pathak, Cassidy Hardin, Francesco Visin, Jiageng Zhang, Kathleen Kenealy, Qin Yin, Olivier Lacombe, Armand Joulin, Tris Warkentin, and Adam Roberts. 2025a. T5Gemma 2: Seeing, reading, and understanding longer. *arXiv preprint arXiv:2512.14856*.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025b. Qwen3 Embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Guoliang Zhao, Ruobing Xie, An Wang, Shuaipeng Li, Huaibing Xie, and Xingwu Sun. 2026. Self-distillation for multi-token prediction. *arXiv preprint arXiv:2603.23911*.

A Summary of Hyperparameters

The main hyperparameters are:

- Source and target length: 3,072 source tokens for both datasets; 384 target tokens for WikiLingua; 256 target tokens for LR-Sum.
- Generation: greedy decoding; at most 256 new tokens; no-repeat trigram blocking.
- Adapter: selected layers $[-1, -4, -8, -12]$; salience gate; 4-layer EncStack; 16 global memory tokens.
- Decoder: 8 layers; hidden size 1024; 16 heads; FFN size 4096; dropout 0.1.
- LoRA: encoder adapters are used during Phase 2.
- WikiLingua MTP: 4 heads; CE weight 0.6; KL weight 0.2; head weights $[1.0, 0.8, 0.5, 0.25]$.

B Additional Experimental Results

The current project snapshot includes the main quality and length results, plus the WikiLingua speed and MTP results reported in this report. It does not include a complete adapter ablation. For that reason, this report does not include an ablation table.

C Additional Qualitative Examples

C.1 Representative Case

The following example comes from test item `test_001852`. The source is a Vietnamese WikiHow-style article about treating hidden pimples. We show a source excerpt for space.

Source (excerpt). Tùy vào loại da và mức độ nghiêm trọng của mụn ẩn mà bạn nên cân nhắc giữa phương pháp chườm ấm với chườm lạnh (chườm đá viên). Chườm ấm có thể giúp làm khô mụn ẩn và dẫn lưu mụn; trong khi đó, chườm lạnh giúp giảm sưng đau tại vị trí mụn. Không nên chườm đá trong giai đoạn đầu của mụn ẩn. Nếu là mụn lớn, viêm nghiêm trọng hơn hoặc đau hơn, bạn nên chườm ấm để rút dịch trong mụn ra. Bạn có thể tạo miếng chườm ấm đơn giản bằng cách nhúng khăn sạch vào nước ấm hoặc nước nóng.

Reference. Chườm ấm hoặc chườm lạnh. Làm mặt nạ táo và mật ong. Dùng tinh dầu tràm trà. Thoa lô hội.

LLM2Seq-Llama. Dùng gel lô hội. Thoa sữa rửa mặt nhẹ dịu. Dù bằng dầu tràm trà. Dưỡng ẩm cho da.

LLM2Seq-Qwen. Chườm đá viên lên mụn. Chườm gạc ấm lên mụn để giảm đau và giảm đau. Sử dụng tinh dầu tràm trà để giảm viêm và giảm chất lỏng. Sử hưởng dầu hạt lô hội để xử lý các vấn đề của mụn có thể làm giảm mụn trứng cá.

T5Gemma. Chườm ấm hoặc chườm lạnh. Thử dùng táo và mật ong để điều trị mụn ẩn. Dùng tinh dầu tràm trà nếu bạn không có các phương pháp khác. Sử dụng lô hội (nha đam) cho hiệu quả tốt nhất, nhưng hãy cẩn thận với những tác động phụ của nó khi sử dụng thường xuyên.

Comment. The three systems recover some topical-treatment actions. LLM2Seq-Llama is short and misses the warm/cold compress step. LLM2Seq-Qwen covers the compress and tea-tree oil ideas, but it repeats phrases and contains awkward wording. T5Gemma is more complete and fluent, but also longer.

C.2 Failure Case

The following example comes from test item `test_002672`. The source is a Vietnamese WikiHow-style article about crying and emotional expression. We show a source excerpt for space.

Source (excerpt). Bạn đã được dạy rằng những người dũng cảm thì thường không khóc? Những người được dạy cách kiềm chế và giấu kín cảm xúc từ bé thường gặp khó khăn trong việc biểu lộ cảm xúc khi họ lớn lên. Nhưng khóc là một phần thiết yếu của cuộc sống giúp cải thiện tâm lý. Khóc có thể là biểu hiện của nỗi buồn, sự đau đớn, nỗi sợ, hạnh phúc, hay thậm chí là niềm vui.

Reference. Quên hết những gì đã biết về khóc. Hiểu được lợi ích của khóc. Tìm hiểu lý do tại sao bạn phải kiềm chế cảm xúc. Cho phép chính mình khóc cũng là một cách chăm sóc bản thân tốt hơn.

LLM2Seq-Llama. Nhận biết cảm giác tội lỗi và hành động. Xác định những suy nghĩ tiêu cực trong cuộc sống của bạn. Tìm kiếm sự trợ giúp chuyên nghiệp.

LLM2Seq-Qwen. Hãy hiểu rằng cảm xúc của bạn là bình thường. Hãy làm gì đó. Hãy để bản thân bị trầm cảm. Hãy cho phép bản thân cảm nhận sự khóc. Hãy nhớ rằng việc phải là người luôn bộc lộ bản thân. Hãy cười ra khỏi vùng an toàn của bạn. Hãy vui vẻ và đau khổ.

T5Gemma. Khóc là một cách tự nhiên để thể hiện cảm xúc. Khóc không chỉ dành riêng cho trẻ em, mà còn có ích lợi đối với người lớn tuổi như bạn. Cố gắng khóc khi cần thiết và đừng ngại ngùng nếu mình đã bắt đầu rơi nước mắt. Hãy nhớ rằng việc biểu lộ những cảm xúc của bản thân sẽ giúp cải thiện tâm trạng. Thử tưởng tượng về thời điểm trước đây khi bạn bật khóc vì điều gì đó.

Comment. This example shows that LLM2Seq-Qwen can preserve conciseness but may produce wrong or unnatural phrases. This motivates future factuality and human evaluation. It also shows why automatic overlap and length statistics must be read together with output inspection.

D Demo Interface Details

The demo provides a source text box, a decoding-mode selector, and a maximum-token control. It reports the generated summary, latency, generated tokens, and tokens per second. In verified MTP mode it additionally reports accepted-token statistics.

E Architecture Details

The adapter uses token-wise layer fusion, a gated residual projection, token salience gating, EncStack memory refinement, and learned global memory tokens. The EncStack directly applies standard Transformer encoder refinement. Layer fusion, token salience gating, and learned global memory tokens adapt common conditioning ideas. These include contextual layer mixing, feature modulation, and continuous prompt tokens. The decoder attends to the final memory M through cross-attention in every layer.

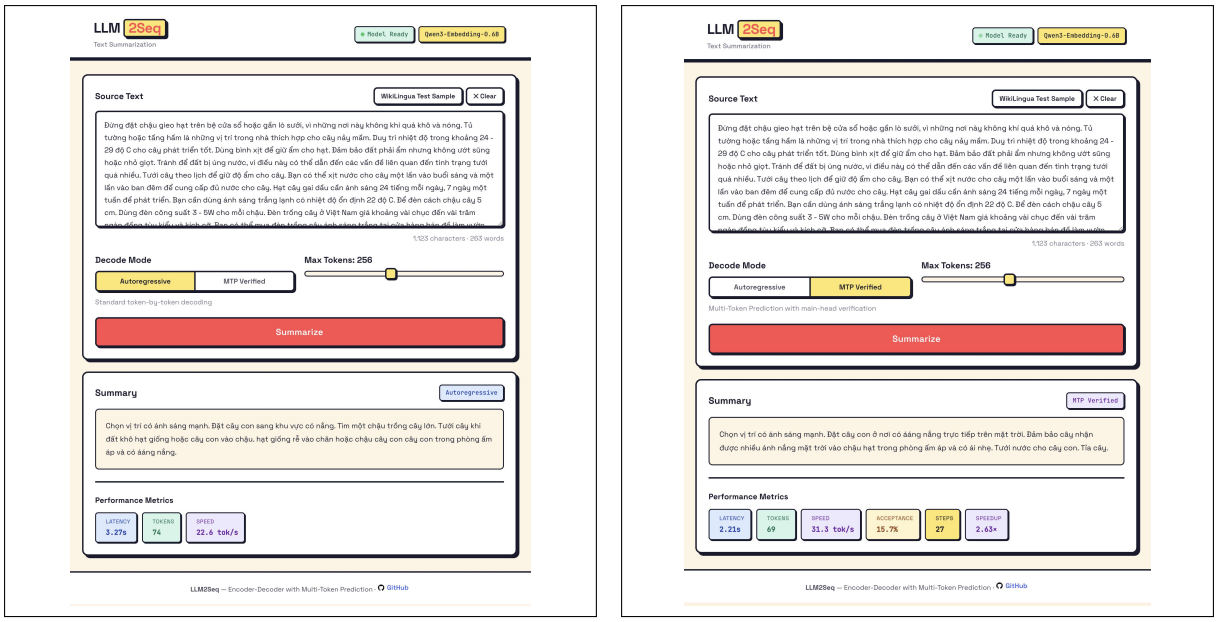

Figure 3: LLM2Seq web demo on the same WikiLingua sample. Left: standard autoregressive decoding. Right: verified MTP decoding.